



# JRCAI

مركز الأبحاث المشترك للذكاء الاصطناعي  
Joint Research Center for AI

**SDAIA | KFUPM**

**King Fahd University of Petroleum and Minerals (KFUPM)**

## ASSIGNMENT COVER SHEET

| Student Name          | Student ID |
|-----------------------|------------|
| Madawi Saleh Alshebil | 3246595    |

### Assignment Details

|                                                                                                                                                                                                                                                                                                                                                                                                                               |                                |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| To be marked by: Dr. Safwan Mahmood Al-Selwi                                                                                                                                                                                                                                                                                                                                                                                  |                                |
| Date submitted: 22/7/2026                                                                                                                                                                                                                                                                                                                                                                                                     | Submission Due Date: 24/7/2026 |
| <b>Submission of this assignment agrees to the following:</b><br><br><i>I understand that the piece of work submitted will be considered as the final and complete version of my assignment of which I am the sole author. I understand both the meaning and consequences of plagiarism and that my work has been appropriately attributed unless otherwise stated. I have not knowingly allowed another to copy my work.</i> |                                |

### Assignment Feedback - Lecturer Use Only

|                            |                                 |
|----------------------------|---------------------------------|
| First Marker Feedback:     |                                 |
| Signature of first marker: | <b>% Mark</b><br>(Un-moderated) |

## 1. Introduction

This report presents an analysis of a small customer purchase dataset containing 15 customer records. The data includes CustomerID, Age, Income, Gender, ProductCategory, and whether the customer made a purchase during a promotional period.

Python was used as the main tool for data preprocessing, exploratory analysis, Decision Tree modelling, model evaluation, and visualization. Weka Web was then used to repeat the preprocessing and classification process as an additional comparison. The main objective was to identify simple patterns in customer purchasing behaviour and develop a Decision Tree model that predicts whether a customer will make a purchase.

## 2. Dataset Description

The dataset contained 15 instances and six attributes. Three attributes were numeric, and three were categorical. The target variable used for prediction was Purchased, which contained the two possible outcomes, yes and no.

*Table 1. Column Description*

| Variable        | Description                                         |
|-----------------|-----------------------------------------------------|
| CustomerID      | Unique identifier for each customer                 |
| Age             | Customer's age                                      |
| Income          | Customer's annual income                            |
| Gender          | Male or Female                                      |
| ProductCategory | Product category associated with the customer       |
| Purchased       | Whether the customer purchased during the promotion |

### 3. Data Preprocessing Using Python

The dataset was first loaded into Python using Pandas. The missing value check showed that Age contained two missing values and Income contained two missing values. Because the dataset contained only 15 records, the incomplete records were not deleted. Instead, the missing values were calculated and rounded to whole numbers because the original Age and Income values were represented as integers. The missing Age values were replaced with 34, while the missing Income values were replaced with 47,154. After the replacement, the Age and Income columns were converted to integer data types.

The categorical variables were then converted into numerical values. Gender was encoded as Female=0 and Male=1, while Purchased was encoded as No=0 and Yes=1. ProductCategory was transformed using one-hot encoding because its categories did not have a meaningful numerical order. This produced four binary columns representing Appliances, Clothing, Electronics, and Furniture.

```
# Counting missing values in every column
print("Missing values in each column:")
print(df.isnull().sum())

# Counting duplicated rows
print("\nNumber of duplicated rows:")
print(df.duplicated().sum())

Missing values in each column:
CustomerID    0
Age            2
Income         2
Gender         0
ProductCategory 0
Purchased      0
dtype: int64

Number of duplicated rows:
0
```

Figure 1. Missing-value and duplicate-row inspection

```
# Cleaning the dataset

df_clean = df.copy() # Creating a cleaned copy of the dataset
df_clean["Age"] = df_clean["Age"].fillna(round(df_clean["Age"].mean()),astype(int))
df_clean["Income"] = df_clean["Income"].fillna(round(df_clean["Income"].mean()),astype(int))
display(df_clean)

...
CustomerID Age Income Gender ProductCategory Purchased
0         1  22  30000   Male      Electronics      Yes
1         2  34  45000  Female      Clothing       No
2         3  34  47154  Female      Electronics    Yes
3         4  45  60000   Male      Furniture     Yes
4         5  23  35000  Female      Electronics    No
5         6  40  55000   Male      Appliances    Yes
6         7  31  40000  Female      Furniture     No
7         8  27  48000   Male      Clothing     Yes
8         9  52  65000  Female      Appliances    Yes
9        10  38  47000   Male      Clothing     No
10       11  26  37000  Female      Furniture     Yes
11       12  34  47154   Male      Electronics    Yes
```

Figure 2. Cleaned dataset

```
# Encoding the categorical variables

df_encoded = df_clean.copy()

df_encoded["Gender"] = df_encoded["Gender"].map({"Female": 0, "Male": 1})
df_encoded["Purchased"] = df_encoded["Purchased"].map({"No": 0, "Yes": 1})
df_encoded = pd.get_dummies(df_encoded, columns=["ProductCategory"], dtype=int)
display(df_encoded)

CustomerID Age Income Gender Purchased ProductCategory_Appliances ProductCategory_Clothing ProductCategory_Electronics ProductCategory_Fur
0         1  22  30000     1         1                0                0                1
1         2  34  45000     0         0                0                1                0
2         3  34  47154     0         1                0                0                1
3         4  45  60000     1         1                0                0                0
4         5  23  35000     0         0                0                0                1
5         6  40  55000     1         1                1                0                0
6         7  31  40000     0         0                0                0                0
7         8  27  48000     1         1                0                1                0
8         9  52  65000     0         1                1                0                0
9        10  38  47000     1         0                0                1                0
10       11  26  37000     0         1                0                0                0
```

Figure 3. Encoded dataset

## 4. Basic Data Analysis Using Python

### 4.1 Mean Age and Income

After cleaning and replacing the missing values, the calculated mean customer Age was 33.87 years. This indicates that the customers in the dataset were approximately 34 years old on average. The calculated mean annual customer Income was 47,153.87, which is approximately 47,154.

```
# Calculating the mean of Age and Income
print(f"Mean Age: {df_encoded['Age'].mean():.2f}")
print(f"Mean Income: {df_encoded['Income'].mean():,.2f}")

Mean Age: 33.87
Mean Income: 47,153.87
```

Figure 4. Python calculation of Age and Income

### 4.2 Histogram Analysis

Two histograms were created to examine the distributions of customer Age and annual Income. Customer ages ranged from 22 to 52 years, with most customers concentrated between their late twenties and early forties. The histogram did not show any clearly extreme age outliers. However, the results should be interpreted cautiously because the dataset contains only a small number of customers.

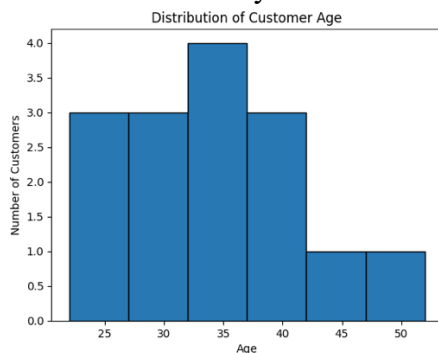


Figure 5. Age histogram

Income values ranged from 30,000 to 65,000. Most customers had annual Incomes between approximately 35,000 and 60,000. The histogram did not show any severe or clearly isolated Income outliers. Similar to the Age distribution, the small sample size limits the conclusions that can be drawn from the histogram.

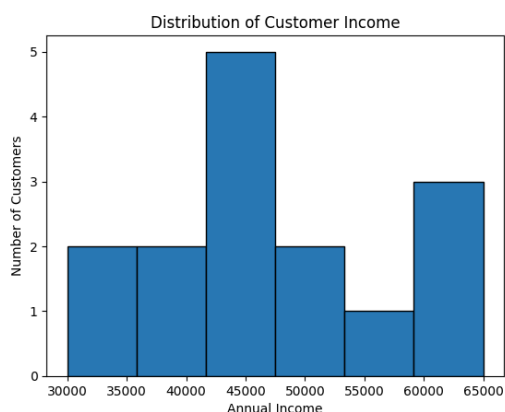


Figure 6. Income histogram

## 5. Decision Tree Modeling

A Decision Tree classification model was developed to predict whether a customer would make a purchase during the promotional period. A Decision Tree creates a sequence of decision rules based on the input attributes and assigns each customer to either Yes or No purchase class.

Age, Income, Gender, and the encoded ProductCategory variables were used as the model inputs, while Purchased was used as the target variable. CustomerID was excluded because it is only an identifier and does not provide meaningful information about purchasing behavior. The model was created using the following settings, where the maximum depth was limited to one level to produce a simple and interpretable tree and reduce overfitting on the small dataset.

```
# Preparing the model variables

X = df_encoded.drop(columns=["CustomerID", "Purchased"]) #input variables
y = df_encoded["Purchased"] #target variable

# Creating the Decision Tree model
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=1, random_state=42)
```

Figure 7. Decision Tree model configuration

## 6. Model Evaluation

### 6.1 Accuracy

The Decision Tree was evaluated using five-fold stratified cross-validation. The dataset was divided into five folds, with approximately 12 records used for training and three records used for testing in each round. The process was repeated until every record had been used for testing.

The model achieved a 73.33% accuracy, which means that the model correctly predicted the purchase outcome for 11 out of 15 customers. The result indicates that the Decision Tree identifies some patterns associated with customer purchasing behavior. However, an accuracy of 73.33% should not be treated as strong evidence that the model will perform equally well on new customers because the dataset is extremely small. Therefore, a larger customer dataset would be needed to obtain a more stable and reliable estimate of the model's performance.

```
# Using five-fold cross-validation to test the model and displaying the accuracy
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.metrics import accuracy_score

# Create five-fold cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Generate predictions
predictions = cross_val_predict(model, X, y, cv=cv)

# Calculate accuracy
accuracy = accuracy_score(y, predictions)

print(f"Decision Tree Accuracy: {accuracy * 100:.2f}%")

Decision Tree Accuracy: 73.33%
```

Figure 8. Decision Tree accuracy

## 6.2 Decision Tree Visualization

The Decision Tree visualization showed that the model used Income for its only split. The root node divided customers according to whether their annual Income was less than or equal to 47,077. Since the tree was limited to a maximum depth of one, no additional variables were used. Therefore, Income was the attribute selected by this fitted model to distinguish between the two purchase outcomes.

Customers with an Income of 47,077 or less followed the left branch and were predicted as No, while customers with an Income greater than 47,077 followed the right branch and were predicted as Yes.

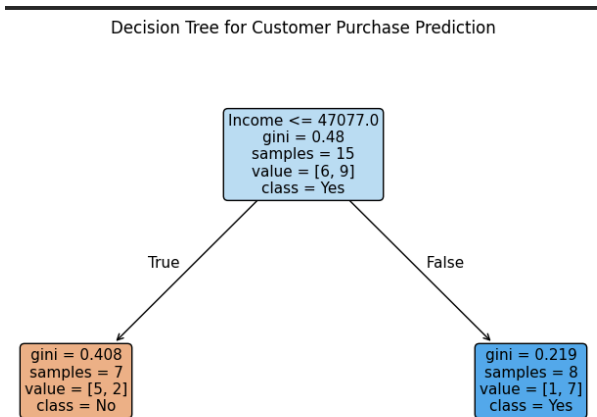


Figure 9. Decision Tree visualization

## 7. Weka Web Implementation

Weka Web was used as an additional application of the same preprocessing and modelling process. A preprocessing pipeline was created to replace the missing Age and Income values using their means, remove CustomerID, and convert Gender and ProductCategory using the Nominal to Integer operation. After preprocessing, the dataset contained 15 instances, five attributes, and no missing values.

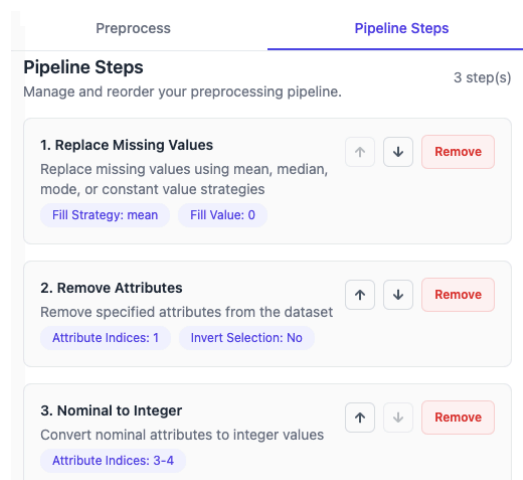
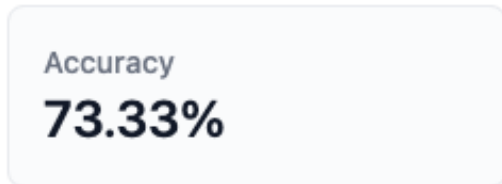


Figure 10. Weka preprocessing pipeline

A Decision Tree classification task was then created using Purchased as the target variable. The model was evaluated using five-fold cross-validation and achieved an accuracy of 73.33%. This documented result was consistent with the Python cross-validation result.

## Performance Metrics



*Figure 11. Weka accuracy*

### 8. Findings

Nine of the 15 customers made a purchase, giving an overall purchase rate of 60%. The fitted Decision Tree used Income for its only split, with a threshold of 47,077. Customers with an Income greater than this value were predicted to purchase, while customers with an Income at or below this value were predicted not to purchase. Although Age, Gender, and ProductCategory were included as model inputs, they were not selected by the fitted one-level tree. This suggests that Income was the most useful attribute for this specific model and dataset.

### 9. Conclusion

The analysis showed that a Decision Tree can be used to identify simple patterns in customer purchasing behaviour. The fitted model used Income for its only decision split and achieved an accuracy of 73.33% using five-fold cross-validation. The model could support marketing teams by helping them identify customer groups that may be more likely to respond to promotional campaigns and direct suitable offers toward them. However, because the dataset contained only 15 records, the results should be interpreted carefully. A larger dataset would be required before using the model for real customer-targeting decisions.